

Spring Boot 가이드

새 도메인이 추가될 때 무엇이 반복되고, 무엇이 새로운가? : 게시판(Post) 도메인

※ 이 문서는 가이드 1 (파일 추적법), 가이드 2 (요청 생명주기)를 먼저 읽고 이어서 보시는 걸 전제로 합니다.

가이드 3. 도메인 확장 (같은 구조 위에 새 도메인이 추가되면 무엇이 반복되고, 무엇이 새로워지는가?)

1. 패턴 재확인 : 구조는 그대로입니다

대상 파일: loginauth-board/src/main/java/loginauth/post/**

post 패키지를 열어보면 domain / dto / exception / repository / service / web 구조가 auth 도메인 때와 완전히 동일합니다. 즉 가이드 1에서 배운 3단계 추적법(필드 목록 확인 → import에서 위치 찾기 → 그 파일로 이동해 반복)을 새로 배울 필요 없이 그대로 다시 쓰시면 됩니다.

새 도메인을 추가한다고 해서 매번 새로운 규칙을 배우는 게 아니라, 같은 패턴이 도메인만 바뀌어 반복된다는 것 자체가 이 구조(계층형 아키텍처)의 핵심 장점입니다.

2. 이번 도메인에서 처음 등장한 개념

2-1. @PathVariable : URL 경로에서 값 꺼내기

```
@GetMapping("/{postId}")
public PostResponse findById(@PathVariable Long postId) {
    return postService.findById(postId);
}
```

{postId}는 URL 경로 자체에 값이 들어간다는 뜻입니다(예: GET /api/posts/5). @PathVariable은 "URL 경로에 박혀 있는 그 값을 꺼내서 매개변수에 넣어달라"는 지시입니다. 지금까지 본 폼 파

라미터나 요청 본문과 달리, URL 자체가 데이터를 담는 방식입니다. 매개변수명(postId)이 {postId}와 정확히 일치해야 자동으로 매칭됩니다.

2-2. @RequestBody : 드디어 등장한 진짜 JSON 입력

```
@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public PostResponse create(
    @Valid @RequestBody PostCreateRequest request,
    Authentication authentication
) {
    return postService.create(request, authentication.getName());
}
```

가이드 1에서 signup()/login()은 @RequestBody가 없어서 폼(application/x-www-form-urlencoded) 데이터를 받았습니다. 여기는 @RequestBody가 붙어 있어서 진짜 JSON 본문으로 요청을 받습니다. 이게 REST API의 표준적인 입력 방식입니다.

signup()과 create()를 나란히 놓고 비교하면 "폼 방식 vs JSON 방식"의 차이가 가장 명확하게 드러납니다.

2-3. @ResponseStatus : 상태코드를 직접 명시

- @ResponseStatus(HttpStatus.CREATED): 성공 시 기본값 200 대신, "새로 만들어졌다"는 의미를 정확히 전달하려고 201로 명시
- @ResponseStatus(HttpStatus.NO_CONTENT): 삭제는 돌려줄 데이터가 없으므로 반환 타입도 void, 상태코드도 204로 명시

2-4. 예외 2 종류와 상태코드 매핑

가이드 2에서 배운 "예외 발생 → 전파 → 처리 담당자 도착" 흐름이 여기서는 예외 종류에 따라 서로 다른 상태코드로 갈라집니다.

- PostNotFoundException: 글이 존재하지 않음 → PostExceptionHandler.notFound() → 404 Not Found
- PostAccessDeniedException: 작성자가 아닌 사람이 수정/삭제 시도 →

PostExceptionHandler.forbidden() → 403 Forbidden

2-5. Spring Data JPA : SQL 없이 저장하기

JDBC/SQL 방식은 이미 다뤘듯이 auth 도메인의 JdbcUserRepository는 SQL을 직접 작성한 클래스였습니다. 이번에는 JPA 방식 구현을 다뤄보겠습니다. 따라서 이번 post 도메인의 PostRepository는 구현체 파일이 어디에도 없습니다. 즉 JpaRepository<Post, Long>를 상속만 하면, Spring Data JPA가 서버 시작 시 자동으로 구현체를 만들어 save(), findById(), findAll(), delete() 같은 기본 CRUD를 전부 제공합니다.

```
public interface PostRepository extends JpaRepository<Post, Long> {  
    List<Post> findAllByIdDesc();    // SQL 한 줄도 안 썼는데 자동 생성됨  
}
```

findAllByIdDesc()처럼 직접 선언한 메서드도, 메서드 이름 자체를 분석해서 "Post를 전부 찾는데 id 기준 내림차순으로 정렬해줘"라는 SQL을 Spring이 자동 생성합니다.

구분	auth 도메인 (JdbcUserRepository)	post 도메인 (PostRepository)
저장 방식	JdbcTemplate + 직접 SQL 작성	Spring Data JPA (SQL 자동 생성)
구현체	직접 클래스 작성 필요	인터페이스만 있으면 Spring이 자동 구현
장점	SQL을 세밀하게 제어 가능	코드량이 훨씬 적고 빠르게 개발 가능

Post.java에는 매개변수 없는 protected 생성자(protected Post() {})도 있습니다. JPA(Hibernate)가 DB에서 데이터를 읽어와 객체를 만들 때, 리플렉션으로 빈 객체부터 만들고 필드를 채워 넣는 방식으로 동작하기 때문에 이 생성자가 필요합니다. 또한 setter 없이 update() 메서드로만 값을 바꾸게 한 것도 눈여겨보기 바랍니다. "언제, 어떻게 값이 바뀌는지"를 코드에 명확히 드러내기 위한 설계입니다.

3. 도메인 간 연결 : Authentication 재사용

새 도메인(post)을 추가했다고 해서 로그인/인증 로직을 다시 만들지 않았습니다.

PostController의 create(), update(), delete()는 전부 Authentication authentication 매개변수를 받는데, 이건 가이드 2에서 배운 그 객체입니다. JwtAuthenticationFilter가 요청마다 쿠키의 JWT를 검증해서 SecurityContextHolder에 채워둔 바로 그 인증 정보입니다.

```
public PostResponse create(@Valid @RequestBody PostCreateRequest request, Authentication authentication) {  
    return postService.create(request, authentication.getName()); // 글쓴이 식별  
}
```

authentication.getName()으로 꺼낸 username이 게시글의 writer로 저장되고, 나중에 수정/삭제 시 "요청자 == 작성자"인지 비교하는 기준이 됩니다(PostService.verifyWriter()). 즉 인증 메커니즘 하나를 만들어두면, 이후 추가되는 모든 도메인이 그걸 공짜로 재사용할 수 있다는 게 필터를 컨트롤러 앞단에 배치해둔 설계의 핵심 이점입니다.

4. 요청 하나의 생명주기 : Post 버전

화살표 규칙은 가이드 2와 동일합니다. ⇄ 는 요청/응답 왕복, → 는 예외가 발생해 위로 튕겨 올라가는 편도 흐름입니다.

Case A. 정상적인 게시글 작성 (POST /api/posts)

1~7. [JwtAuthenticationFilter.java] ⇄ [CookieService.java] / [JwtProvider.java] 등

가이드 2의 Case A (로그인 성공 후 상태 확인)와 완전히 동일한 흐름입니다. 쿠키에서 토큰을 읽고, JwtProvider로 검증하고, SecurityContextHolder에 인증 정보를 등록한 뒤 다음 단계로 진행합니다.

8. [Spring 프레임워크 내부, SecurityConfig.java 참고] ⇄ [Spring 프레임워크 내부] DispatcherServlet

요청 : "/api/posts" 요청이 인증된 사용자만 허용하는 경로인지 확인

응답 : 직전 단계에서 등록된 인증 정보로 통과 판정

9. [Spring 프레임워크 내부] DispatcherServlet ⇄ [PostController.java] create()

요청 : 매핑된 메서드 실행 요청. @RequestBody로 JSON 본문을, Authentication으로 인증 정보를 함께 전달

응답 : create() 메서드 실행 시작

10. [PostController.java] create() ⇔ [PostService.java] create()

요청 : request(PostCreateRequest)와 authentication.getName()(작성자)을 넘기며 생성을 위임

응답 : (내부적으로 11~13번을 거쳐) 완성된 PostResponse를 돌려받음

11. [PostService.java] create() ⇔ [Post.java] new Post(title, content, username)

요청 : 제목·내용·작성자로 새 도메인 객체 생성을 요청

응답 : 새 Post 인스턴스가 만들어짐 (아직 id 없음, DB 미저장 상태)

12. [PostService.java] create() ⇔ [PostRepository.java] save(post)

요청 : 방금 만든 Post 객체 저장을 요청

응답 : Spring Data JPA가 자동 생성한 INSERT 쿼리가 실행되고, id가 채워진 Post 객체가 반환됨

13. [PostService.java] create() ⇔ [PostResponse.java] PostResponse.from(post)

요청 : 저장된 Post 엔티티를 응답 전용 DTO로 변환 요청

응답 : PostResponse 객체가 만들어져 반환됨

14. [PostController.java] ⇔ [Spring 프레임워크 내부] (JSON 직렬화)

요청 : 완성된 PostResponse를 반환. @ResponseStatus(CREATED)로 상태코드도 함께 지정됨

응답 : JSON으로 직렬화되어 201 Created로 최종 전송

전 단계가 ㄹ(왕복)로만 이어집니다. 예외 없이 정상적으로 끝까지 완료되는 흐름입니다.

Case B. 남의 글을 수정하려는 시도 (PUT /api/posts/{postId})

로그인은 되어 있지만, 이 글의 작성자가 아닌 사용자가 수정을 시도하는 경우입니다.

1~8. [JwtAuthenticationFilter.java] 등 Case A와 동일

이 사용자도 정상적으로 로그인되어 있으므로, 필터·인가 단계까지는 Case A와 완전히 동일하게 통과합니다.

9. [Spring 프레임워크 내부] DispatcherServlet ⇄ [PostController.java] update()

요청 : postId(PathVariable), request(PostUpdateRequest), authentication을 함께 전달

응답 : update() 메서드 실행 시작

10. [PostController.java] update() ⇄ [PostService.java] update()

요청 : postId, request, authentication.getName()을 넘기며 수정을 위임

응답 : 정상적인 값을 들고 돌아오지 못함 — 이 지점부터 흐름이 예외로 갈라짐

11. [PostService.java] update() → findPost(postId) ⇄ [PostRepository.java] findById(postId)

요청 : postId로 글을 조회 요청

응답 : 글이 존재하므로 정상적으로 Post 객체 반환 (이 단계는 예외 없이 통과)

12. [PostService.java] update() → verifyWriter(post, username) 내부 실행

post.getWriter().equals(username)을 비교, 글쓴이와 지금 요청한 사람이 다를 것을 확인.

13. [PostService.java] verifyWriter() → [PostAccessDeniedException.java] 생성 — 예외 발생

throw new PostAccessDeniedException() 실행, 정상적인 값 반환이 아니라 예외 객체가 만들어져 즉시 던져짐.

14. [PostService.java] → [PostController.java] update() 예외 전파

verifyWriter()에서 던진 예외가 update() 메서드 밖으로, 다시 호출자였던

PostController.update()로 튀어 오름.

15. [PostController.java] → [Spring MVC 프레임워크 내부] 예외 전파

update() 안에도 이 예외를 잡는 코드가 없어, 예외가 프레임워크 레벨까지 계속 위로 전파됨.

16. [Spring MVC 프레임워크 내부] → [PostExceptionHandler.java] forbidden() 예외 전파 (처리 담당자 탐색)

전파되던 예외의 타입(PostAccessDeniedException)과 일치하는 @ExceptionHandler를 찾아냄.

17. [PostExceptionHandler.java] forbidden() ⇔ [Spring 프레임워크 내부] (응답 직렬화)

요청 : 예외를 넘겨받아 어떤 응답을 만들지 처리를 요청 받음

응답 : `ResponseEntity.status(403).body(Map.of("message", "작성자인 게시글을 수정하거나 삭제할 수 있습니다.))`를 반환 → 403 Forbidden으로 전송

짚을 포인트: 1~10번, 17번은 ㄹ, 11번도 ㄹ(글 조회는 정상 성공), 13~16번만 →(편도)입니다. "글을 찾는 것"과 "권한을 확인하는 것"은 서로 다른 단계이고, 예외는 권한 확인에서만 발생했다는 걸 화살표가 정확히 보여줍니다.

5. Auth 도메인 vs Post 도메인 한눈에 비교

기준	auth 도메인 (signup/login)	post 도메인 (create/update)
입력 방식	폼 데이터 (application/x-www-form-urlencoded)	JSON 본문 (@RequestBody)
응답 방식	302 리다이렉트	데이터(JSON) + 상태코드(201/204)
저장 방식	JdbcTemplate + 직접 SQL	Spring Data JPA (SQL 자동 생성)
REST 여부	아니오 (전통적 폼 처리)	네 (JSON, 데이터 응답, 무상태 모두 충족)
예외 종류	InvalidCredentialsException 1종류	PostNotFoundException / PostAccessDeniedException 2종류

이 표에서 보여주듯, post 도메인은 auth 도메인과 같은 프로젝트 안에 있으면서도 훨씬 더 "전형적인 REST API" 방식으로 설계되어 있습니다. 두 도메인을 나란히 비교하며 읽으면, "REST API란 무엇인가"와 "저장 방식은 왜 다양한가"라는 두 가지 개념을 동시에 체득할 수 있습니다.